

线性分组码及代数译码

peterrolls@qq.com

2026 年 5 月 30 日

1 线性分组码

分组码对信源输入的符号按固定长度进行分组，以“组”为单位进行编码，不同分组之间相互独立。每输入 k 个信源符号，编码输出 n 个信源符号，其中 $n > k$ ，该 n 个符号组成一个“分组”。对于每个分组的 n 个符号，如果每个符号都是 k 个输入符号的线性组合，则该编码称为线性分组码。

1.1 定义

一个长度为 n ，有 2^k 个码字的分组码，当且仅当共 2^k 个码字构成域 $GF(2)$ 上所有 n 维向量组成的向量空间的一个 k 维子空间时被称为线性 (n, k) 码。

1.2 二进制线性分组码成群

一个二进制分组码是线性分组码的充要条件是其任意两个码字的模2和仍然是该分组码的一个码字。

1.3 生成矩阵

由线性分组码的定义，一个 (n, k) 线性码 C 的 n 个码字中，只有 k 个码字承载信息，后 $n - k$ 个码字是由码字生成的冗余信息。所以可以选出 k 个线性独立的码字， $g_0, g_1, \dots, g_{k-1}$ ，使得 C 中的每个码字都是这 k 。将选出的这一组码字一行一个排列，得到 $n \times k$ 的矩阵，这个矩阵称为 (n, k) 线性码的生成矩阵，记为 G ：

$$G = \begin{bmatrix} g_0 \\ g_1 \\ \vdots \\ g_{k-1} \end{bmatrix} = \begin{bmatrix} g_{0,0} & g_{0,1} & \cdots & g_{0,n-1} \\ g_{1,0} & g_{1,1} & \cdots & g_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ g_{k-1,0} & g_{k-1,1} & \cdots & g_{k-1,n-1} \end{bmatrix}$$

1.4 线性分组码编码

将待编码的一组信源序列 $u = [u_0, u_1, \dots, u_{k-1}]$ 与生成矩阵相乘，就完成了线性分组码的编码：

$$v = u \cdot G = [u_0, u_1, \dots, u_{k-1}] \cdot G = \begin{bmatrix} g_0 \\ g_1 \\ \vdots \\ g_{k-1} \end{bmatrix} = u_0 g_0 + u_1 g_1 + \dots + u_{k-1} g_{k-1}$$

1.5 系统码

系统码是信息部分和冗余校验部分在生成码字，也就是编码后有固定位置的码字结构。

一个 (n, k) 线性系统码完全由 $k \times n$ 矩阵 G 确定，其形式如下：

$$G = \begin{bmatrix} g_0 \\ g_1 \\ \vdots \\ g_{k-1} \end{bmatrix} = \begin{bmatrix} P_{k \times n-k} & I_{k \times k} \end{bmatrix}$$

也就是将生成矩阵进行行变换，将最右侧变为一个单位阵，再与信源序列相乘，得到的生成码字就是系统码。其右侧是信源序列本身，左侧是冗余校验部分。

1.6 奇偶校验矩阵

首先考虑 (n, k) 系统码，其生成矩阵为 G ，现使用矩阵的分块构造一个 $(n - k) \times n$ 的矩阵如下：

$$H = \begin{bmatrix} I_{n-k} & P^T \end{bmatrix}$$

则 H 矩阵的行秩为 $n - k$ ，对于任何一个码字， $v = u \cdot G$ 有 $v \cdot H^T = \mathbf{0}_{1 \times (n-k)}$ 。 H 矩阵就是该码的奇偶校验矩阵。

若 (n, k) 线性分组码的奇偶校验矩阵 H 不具有标准的 $[I_{n-k} \ P^T]$ 形式，可以通过对 H 进行初等行变换，使其变为标准形式。然后得到生成矩阵 G ，此时 G 可生成系统码，且 $HG^T = \mathbf{0}$ 。

1.7 向量差错/错误模式

由于信道有噪声，认为噪声也是输出码字形状相同的向量，求出噪声向量，用接收向量减去噪声向量就完成了纠错。

对于二进制码，加法和减法相同，都是符号的翻转，所以认为：

$$e = r + v = [e_0, e_1, \dots, e_{n-1}]$$

为差错向量或错误模式。其中 r 为信道输出端接收到的码字， v 为输入到有噪信道的码字。

1.8 校正子(Syndrome)

计算：

$$r \cdot H^T \triangleq s = [s_0, s_1, \dots, s_{n-k-1}]$$

称 s 为 r 的校正子。

如果 $s = \mathbf{0}_{1 \times (n-k)}$ ，即 r 通过 H 的奇偶校验，则 r 是一个码字，可认为 r 是传输码字；

如果 $s \neq \mathbf{0}_{1 \times (n-k)}$ ，可知 r 不是码字，检测出存在差错。可以自动请求重传（ARQ），也可以进行前向纠错（FEC）。

如果发送 v_1 ，经信道传送后变成了另一个码字 v_2 ， $s = v_2 H^T = \mathbf{0}$ ，则检测不出差错。

1.9 纠错能力

- 码组集中任意两个码字之间距离的最小码距(d_{min})关系着这种编码的检错和纠错能力。
- 为检测出 e 个错码， $d_{min} \geq e + 1$ ；
- 为纠正 t 个错码， $d_{min} \geq 2t + 1$ ；
- 为检测出 e 个错码，同时纠正 t 个错码， $d_{min} \geq t + e + 1$ 且 $e \geq t$ 。

一个最小距离为 d_{min} 的 (n, k) 线性分组码通常表示成 (n, k, d_{min}) 。

1.10 标准阵

标准阵是一个 2^{n-k} 行， 2^k 列的矩阵，它是这样生成的：

1. 初始化第一行：第一行是线性码的 2^k 个码字，按行排列，以全零码字作为第一行的第一个元素；
2. 选择第一个陪集首：从剩余的 $2^n - 2^k$ 个码字中选择一个汉明重量最小的 n 元组作为陪集首，将其放置在0的下方；
3. 生成陪集：将该陪集首加到第一行的每一个码字上，形成新的一行，即该陪集首对应的陪集；
4. 重复操作：继续从未使用的 n 元组中选择陪集首生成陪集，直到所有的 n 元组都被写入表中。

1.11 线性分组码译码

1. 计算 r 的校正子 $s = r \cdot H^T$ ；
2. 确定校正子 s 在标准阵中的陪集首 e ，于是 e 被认定为由信道引起的错误模式；
3. 将接收向量 r 译为码字 $v^* = r + e$ 。

这种方法被称为“校正子译码”或“查表译码”。原则上，这种方法可以用于任意的 (n, k) 线性码。该方案提供最小的译码延时和差错概率，然而，对于很大的 $n - k$ ，这种译码方式需要很大的存储量或复杂的逻辑电路，其实现是不现实的。

2 汉明码

汉明码（Hamming Code）是一种经典的线性分组码，由理查德·汉明（Richard Hamming）于1950年提出。它能够检测并纠正单个比特错误，是纠错编码中效率较高且实现简单的一类码。

汉明码具有特定的参数结构：确定单个参数 m ，即可确定汉明码的参数。

码长	信息位	校验位	最小汉明距离	可纠正错误
$n = 2^m - 1$	$k = 2^m - m - 1$	$r = m$	$d_{min} = 3$	$t = 1$

表 1: 汉明码参数

汉明码是线性分组码的子类，所以其编解码方式和一般线性分组码相同。

2.1 汉明码是一种完全码

以码字为中心，以可纠错数为半径的 n 维“球”空间球内没有其他码字。这样的码是完全码。

数学上，完全码满足：

$$\sum_{i=0}^t \binom{n}{i} = 2^{n-k}$$

其中 n 为码长， k 为信息位长度， t 为可纠正错误数。目前已知的二进制完全码有汉明码，(23,12)Golay码，以及三进制(11,6)码。

2.2 汉明码的译码

除了使用查表译码的方法，由于汉明码只纠正1位错误，还可以用以下方式译码：

1. 计算校正子 $s = H \cdot r^T$ ；
2. 如果校正子为0，不操作；
3. 如果校正子不为0，将其与校验矩阵H进行匹配比较，校正子匹配到的列号对应错误位置。翻转错误位置完成纠错。

3 循环码

循环码也是线性分组码的一个子类。

一个 (n, k) 线性码 C ，如果每一个码字的循环一位仍是该码 C 的码字，则称其为循环码(Cyclic Code)。

3.1 循环码的多项式表示

将码字 $v = [v_0, v_1, \dots, v_{n-1}]$ 的各个分量看成如下多项式的系数：

$$v(x) = v_0 + v_1x + v_2x^2 + \dots + v_{n-1}x^{n-1}$$

若 $v_{n-1} \neq 0$ ，则 $v(x)$ 的次数为 $n-1$ ，若 $v_{n-1} = 0$ ，则 $v(x)$ 的次数小于 $n-1$ 。则：

$$v^{(i)}(x) = v_{n-i} + v_{n-i+1}x + \dots + v_{n-i}x^{i-1} + v_0x^i + v_1x^{i+1} + \dots + v_{n-i-1}x^{n-1}$$

这里 $v^{(i)}(x) = x^i \cdot v(x) \bmod x^n + 1$ ，从而码多项式 $v^{(i)}(x)$ 是 $x^i \cdot v(x)$ 除以 $x^n + 1$ 所得的余式。

3.2 循环码次数最低的非零码多项式唯一且常数项为1

循环码 C 中次数最低的非零码多项式是唯一的，且最低次数的码多项式的常数项为1。

3.3 循环码的码字都是最低次数非零多项式的倍式

设 $g(x) = 1 + g_1x + \dots + g_{n-k-1}x^{n-k-1} + x^{n-k}$ 是 (n, k) 循环码的最低次数非零多项式。则次数小于或等于 $n-1$ 的二进制多项式是码多项式的充要条件是该多项式是 $g(x)$ 的倍式。

3.4 $(n-k)$ 次 $x^n + 1$ 因式多项式存在，就可以生成循环码

如果 $g(x)$ 是次数为 $(n-k)$ 的多项式且为 $x^n + 1$ 的因式，则 $g(x)$ 生成一个 $(n-k)$ 循环码。

$x^n + 1$ 的任何一个次数为 $(n-k)$ 的多项式，都可以生成一个 $(n-k)$ 循环码。对不同的生成多项式，其生成的循环码码集不同。

3.5 循环码编码

由于 (n, k) 循环码每一个码多项式可表示为：

$$v(x) = u(x)g(x) = (u_0 + u_1x + \dots + u_{k-1}x^{k-1})g(x)$$

这里的 u_i 为待编码的信息位， $g(x)$ 为生成多项式。这种编码方式得到的码通常不是系统码。

3.6 循环码的系统码编码

系统码编码步骤如下：

1. 用 x^{n-k} 乘以消息 $u(x) = u_0 + u_1x + \cdots + u_{k-1}x^{k-1}$ ；
2. 计算 $x^{n-k}u(x)$ 关于 $g(x)$ 的余式 $b(x) = b_0 + b_1x + \cdots + b_{n-k-1}x^{n-k-1}$ ；
3. 联合 $b(x)$ 和 $x^{n-k}u(x)$ ，得到系统码 $b(x) + x^{n-k}u(x)$ 。

3.7 生成矩阵

考虑一个 (n, k) 循环码 C ，其生成多项式 $g(x) = g_0 + g_1x + \cdots + g_{n-k}x^{n-k}$ ，($g_0 = g_{n-k} = 1$)，则 k 个多项式 $g(x), xg(x), \cdots, x^{k-1}g(x)$ 的系数向量是线性独立的，由它们可以构成码的生成矩阵：

$$G = \begin{bmatrix} g_0 & g_1 & \cdots & g_{n-k} & 0 & \cdots & 0 \\ 0 & g_0 & \cdots & g_{n-k-1} & g_{n-k} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & g_0 & g_1 & \cdots & g_{n-k} \end{bmatrix}$$

通常 G 不是系统形式。可通过行变换将其转化成系统形式，即右侧是一个分块的单位阵。

3.8 校验矩阵

前已知道，生成多项式是 $x^n + 1$ 的因式，即：

$$x^n + 1 = g(x)h(x)$$

其中 $h(x)$ 是具有如下形式的 k 次多项式：

$$h(x) = h_0 + h_1x + \cdots + h_kx^k$$

这里 $h_0 = h_k = 1$ ，循环码 C 的校验矩阵可由 $h(x)$ 得到：

$$H \triangleq \begin{bmatrix} h_k & h_{k-1} & \cdots & h_0 & 0 & \cdots & 0 \\ 0 & h_k & \cdots & h_1 & h_0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & h_k & h_{k-1} & \cdots & h_0 \end{bmatrix}$$

3.9 循环码译码

循环码的译码步骤与线性码是一致的，计算接收矢量的伴随式，根据伴随式判断是否有错，有错时进而判断错误图样并进行修正。由于循环码的结构使得它的译码可以较线性码简单且易于实现。

由于伴随式循环移位在模 $g(x)$ 下的结果正好和码字的循环移位相对应，这使得对循环码字的任何一位的译码都是循环等价的。Meggitt译码方法利用这点，预存一些错误图样，如果接收码字自身或循环移位后的伴随式与错误图样对应，则将错误图样对应的错误位数对应移动几位就是码字的错误位。若没有匹配，则说明有超过纠错能力的错误。

Meggitt译码方法的步骤如下：

1. 接收与缓存：接收码字 $R(x)$ 输入缓冲寄存器和伴随式计算电路，经 n 次移位后，缓冲寄存器存储完整码字，伴随式电路计算得到初始伴随式 $S_0(x)$ ；
2. 伴随式检测：若 $S_0(x)$ 匹配错误图样，则触发纠错信号；

3. 循环移位与纠错：若未匹配，伴随式电路继续循环移位，每次移位后检查新的伴随式。移位次数 i 对应错误位置 $n - i$ ；
4. 输出正确码字：纠错完成或判定无法纠错后，从缓冲寄存器中输出修正后的码字。

4 二进制BCH码

BCH码是一类重要的循环码，能纠正多个随机错误。在前面的讨论中，我们所做的是先构造码，然后评估其纠错能力；BCH码中，我们先确定期望的纠错个数，然后构造码。

对于任意正整数 $m(m \geq 3)$ ，存在具有如下参数的二进制BCH码：

分组长度	奇偶校验位数目	最小距离
$n = 2^m - 1$	$n - k \leq mt$	$d_{min} \geq 2t + 1$

表 2: 二进制BCH码参数

该码可纠正 t 个或小于 t 个差错的任意组合。纠单个错误的二进制BCH码就是汉明码。

4.1 本原二进制BCH码的生成多项式

考虑 $GF(2^m) = \{0, 1, \alpha, \dots, \alpha^{2^m-2}\}$ ，设 α 为本原元。码长为 $2^m - 1$ ，纠 t 个差错的BCH码的生成多项式 $g(x)$ 就是以 $\alpha, \alpha^2, \alpha^3, \dots, \alpha^{2t}$ 为根的最低次数多项式(Lowest Degree Polynomial)。对于 $i = 1, 2, \dots, 2t$ ，有 $g(\alpha^i) = 0$ 。

令 $\phi_i(x)$ 为 α^i 的极小多项式，则 $g(x)$ 必为 $\phi_1(x), \phi_2(x), \dots, \phi_{2t}(x)$ 的最小公倍式：

$$g(x) = \text{LCM}\{\phi_1(x), \phi_2(x), \dots, \phi_{2t}(x)\}$$

由于每一个偶次幂都与其前面某个奇次幂有相同的极小多项式，从而：

$$g(x) = \text{LCM}\{\phi_1(x), \phi_3(x), \dots, \phi_{2t-1}(x)\}$$

这种 $g(x)$ 中有一个根为本原元，由这种生成多项式构成的二进制BCH码是二进制本原BCH码，其码长为 $2^m - 1$ 。

4.2 一般二进制BCH码的生成多项式

选取有限域 $GF(2^m) = \{0, \alpha^0 = 1, \alpha^2, \dots, \alpha^{2^m-2}\}$ (α 为本原元)的一个子集，取 $\beta = \alpha^\rho$ ($1 \leq \rho < 2^m - 1$)，则 β 的阶为：

$$n = \frac{2^m - 1}{\text{GCD}(2^m - 1, \rho)} | 2^m - 1$$

所以 $\beta, \beta^2, \beta^3, \dots, \beta^{2t}$ 都是 $x^n + 1$ 的根。

设 $\phi_1(x), \phi_2(x), \phi_3(x), \dots, \phi_{2t}(x)$ 分别是 $\beta, \beta^2, \beta^3, \dots, \beta^{2t}$ 的极小多项式，则这也可以获得一个生成多项式：

$$g(x) \triangleq \text{LCM}(\phi_1(x), \phi_2(x), \phi_3(x), \dots, \phi_{2t}(x)) | (x^n + 1)$$

这样， $g(x)$ 是一个码长为 n 的循环码的生成多项式，这里码长 n 的值是 β 的阶数，也是 $2^m - 1$ 的因子。

如果 β 不是 $GF(2^m)$ 的本原元，则 $n \neq 2^m - 1$ ， $g(x)$ 生成的是二进制非本原BCH码。

4.3 二进制BCH码的系统码编码

同循环码一样，二进制BCH码在求出生成多项式后，按照循环码的方式进行系统码编码：

1. 确定码参数：选择 m ，计算 $n = 2^m - 1$ ，设计距离 $d = 2t + 1$ 。对非本原BCH码，需要另选择码长 $n|2^m - 1$ ；
2. 构造生成多项式 $g(x)$ ：构造本原或非本原BCH码的生成多项式；
3. 信息多项式 $u(x)$ ：将 k 位信息比特转换为多项式 $u(x)$ ；
4. 计算余式 $r(x)$ ： $r(x) = x^{n-k} \cdot u(x) \bmod g(x)$ ；
5. 组合码字： $c(x) = x^{n-k} \cdot u(x) + r(x)$ 。

4.4 二进制本原BCH码译码

奇偶校验矩阵唯一确定一种二进制本原BCH码：

$$H = \begin{bmatrix} 1 & \alpha & \alpha^2 & \alpha^3 & \cdots & \alpha^{n-1} \\ 1 & (\alpha^2) & (\alpha^2)^2 & (\alpha^2)^3 & \cdots & (\alpha^2)^{n-1} \\ 1 & (\alpha^3) & (\alpha^3)^2 & (\alpha^3)^3 & \cdots & (\alpha^3)^{n-1} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\alpha^{2t}) & (\alpha^{2t})^2 & (\alpha^{2t})^3 & \cdots & (\alpha^{2t})^{n-1} \end{bmatrix}$$

与循环码一样，设定以下参数：

- 发送码字： $v(x) = v_0 + v_1x + v_2x^2 + \cdots + v_{n-1}x^{n-1}$ ；
- 接收码字： $r(x) = r_0 + r_1x + r_2x^2 + \cdots + r_{n-1}x^{n-1}$ ；
- 错误图样： $e(x) = e_0 + e_1x + e_2x^2 + \cdots + e_{n-1}x^{n-1}$ ；

有： $r(x) = v(x) + e(x)$ ，校正子（伴随式）是一个 $1 \times 2t$ 的向量：

$$s = [s_1, s_2, \cdots, s_{2t}] = r \cdot H$$

4.4.1 错误位置多项式

定义错误位置多项式：

$$\sigma(x) = (1 + \beta_1x)(1 + \beta_2x) \cdots (1 + \beta_vx) = \sigma_0 + \sigma_1x + \sigma_2x^2 + \cdots + \sigma_vx^v$$

其中 $v \leq t$ 为实际错误数。 $\sigma(x)$ 的根是错误位置的倒数。求得这个错误位置的方法有“钱搜索”方法。

4.4.2 钱搜索(Chien Search)

求出 $\sigma(x) = \sigma_0 + \sigma_1x + \sigma_2x^2 + \cdots + \sigma_vx^v$ 后，将 $\alpha_0^{-1} = 1, \alpha_1^{-1}, \alpha_2^{-1}, \cdots, \alpha_{2^m-2}^{-1}$ 分别代入 $\sigma(x)$ ，求出所有满足 $\sigma(\beta_u^{-1}) = 0$ 的 β_i ，即可知道是第 i 位发生了错误。有无数个根也以此类推，将错误位置的接收码翻转过来即可。

4.4.3 译码流程

二进制本原BCH码的译码步骤如下：

1. 计算校正子： $s = [s_1, s_2, \dots, s_{2t}] = r \cdot H^T$ ；
2. 使用Berlekamp-Massey综合算法，输入校正子序列，得到最短线性移位寄存器 $\langle \sigma(x), v \rangle$ ；
3. 采用Chien搜索算法计算错误位置，进而得到错误图样，实际上可以直接得到错误位置；
4. 计算 $v(x) = v(x) + e(x)$ ，完成译码。

5 RS码

Reed-Solomon码是一种非二进制的BCH码。

将二进制BCH码的伽罗华域改为 $\text{GF}(q)$ ，其中 q 为一个素数的幂，且 $q \neq 2$ ，码元符号和码的生成多项式 $g(x)$ 的根同取自 $\text{GF}(q)$ 的BCH码称为Reed-Solomon码，简称RS码。通常取 $q = 2^m (m > 1)$ 。

符号取自 $\text{GF}(q)$ ，纠 t 个差错的RS码具有如下参数：

分组长度	奇偶校验符号数	维数	最小距离
$q - 1$	$n - k = 2t$	$k = q - 1 - 2t$	$d_{\min} = 2t + 1$

表 3: RS码参数

由于 $g(x)$ 本身是一个码多项式，因而其 $2t + 1$ 个系数均不能为0，否则 $g(x)$ 对应的码字重量小于 $2t + 1$ 。

RS码的奇偶校验矩阵如下：

$$H = \begin{bmatrix} 1 & \alpha & \alpha^2 & \alpha^3 & \dots & \alpha^{q-2} \\ 1 & (\alpha^2) & (\alpha^2)^2 & (\alpha^2)^3 & \dots & (\alpha^2)^{q-2} \\ 1 & (\alpha^3) & (\alpha^3)^2 & (\alpha^3)^3 & \dots & (\alpha^3)^{q-2} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\alpha^{2t}) & (\alpha^{2t})^2 & (\alpha^{2t})^3 & \dots & (\alpha^{2t})^{q-2} \end{bmatrix}$$

5.1 RS码的生成多项式

RS码的生成多项式的求法和二进制本原BCH码的生成多项式一样。在这里记录一个例子：

令 α 为基于本原多项式 $p(X) = 1 + X + X^6$ 构造的伽罗华域 $\text{GF}(2^6)$ 中的本原元。考虑符号取自 $\text{GF}(2^6)$ 、纠正3个错误的RS码。

该码的生成多项式 $g(X)$ 以 $\alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6$ 为其全部的根；因此：

$$\begin{aligned} g(X) &= (X + \alpha)(X + \alpha^2)(X + \alpha^3)(X + \alpha^4)(X + \alpha^5)(X + \alpha^6) \\ &= \alpha^{21} + \alpha^{10}X + \alpha^{55}X^2 + \alpha^{43}X^3 + \alpha^{48}X^4 + \alpha^{59}X^5 + X^6 \end{aligned} \quad (1)$$

该码是 $\text{GF}(2^6)$ 上纠3个错的(63, 57)RS码。

5.2 RS码的系统码编码

RS码的系统码编码也和二进制BCH码一样。设信息多项式为 $u(X) = u_0 + u_1x + \dots + u_{k-1}x^{k-1}$ ，则系统码为 $c(x) = x^{n-k} \cdot u(x) + r(x)$ ，其中 $r(x) = x^{n-k} \cdot u(x) \bmod g(x)$ 。

5.3 RS码的Berlekamp译码

译码由以下步骤组成：

1. 计算伴随式 $s = [s_1, s_2, \dots, s_{2t}] = r \cdot H^T$
2. 计算错误位置多项式：使用Berlekamp-Massey综合算法，输入校正子序列，得到最短线性移位寄存器 $\langle \sigma(x), v \rangle$
3. 计算错误值：使用Forney算法求出错误值。
4. 得出错误图样，完成纠错

错值多项式这样定义：

$$\omega(x) \equiv s(x)\sigma(x) \bmod x^{2t}$$

其中 $\sigma(x) = (1 - \beta_1 x)(1 - \beta_2 x) \cdots (1 - \beta_v x)$ 是 $r(x)$ 的错误位置多项式。

Forney算法指出，错值可由下式给出：

$$y_p = \frac{\omega(\beta_p^{-1})}{\beta_p \prod_{j \neq p} (1 - \beta_j \beta_p^{-1})} = \frac{-\omega(\beta_p^{-1})}{\sigma'(\beta_p^{-1})} \quad p = 1, 2, \dots, v$$

其中 $\sigma'(x)$ 是 $\sigma(x)$ 的导数。